

公司車輛管理系統

114-2 軟體框架設計 期末專題報告書

專題成員

學號	姓名
D1210799	王建葦
D1249623	陳稚翔

一、專案背景

許多企業擁有公務車隊，用於員工出差、外賓接待或日常業務往來。然而，在缺乏系統化管理的情況下，往往面臨以下問題：

- 車輛使用衝突：多位員工同時申請同一輛車，缺乏協調機制。
- 保養遺漏：無法追蹤各車輛的保養週期，導致車況惡化。
- 資訊不透明：管理員無法即時掌握車輛目前的使用狀況。
- 稽核困難：借用紀錄分散，難以進行事後查詢與責任釐清。

本專題設計並實作一套「公司車輛管理系統」，透過清晰的軟體框架設計，提供結構化的車輛申請、審核、追蹤與健康管理功能，並以物件導向設計原則為核心架構依據。

二、系統目標與範圍

2.1 系統目標

- 建立完整的車輛資產管理機制，包含基本資訊、保養紀錄與使用歷程。
- 提供角色導向的存取控制，區分管理員與員工的操作權限。
- 實現車輛借用申請、審核、出車與還車的完整工作流程。

2.2 範圍界定

項目	涵蓋範圍
使用者管理	帳號建立、角色指派、登入驗證
車輛管理	車輛 CRUD、狀態追蹤

項目	涵蓋範圍
申請工作流程	申請 → 審核 → 出車 → 還車
保養管理	保養紀錄新增、到期提醒
報表與稽核	借用歷程查詢、匯出

三、系統角色與使用案例

3.1 角色定義

角色	說明	主要職責
管理員 (Admin)	車隊管理人員，具有最高權限	管理車輛資料、審核申請、查看全部紀錄
員工 (Employee)	一般公司員工	提交借車申請、查看個人借用紀錄

3.2 使用案例

UC-01 申請借用車輛（員工）

欄位	說明
參與者	員工
前置條件	員工已登入系統
主流程	1. 選擇欲使用日期與時段；2. 系統顯示可用車輛清單；3. 員工選擇車輛並填寫用途說明；4. 員工送出申請；5. 申請進入待審核佇列
替代流程	若所選時段無可用車輛，系統顯示衝突提示
後置條件	借用申請狀態為「待審核」

UC-02 審核借用申請（管理員）

欄位	說明
參與者	管理員
前置條件	存在「待審核」狀態的借用申請件
主流程	1. 管理員查看申請清單；2. 選擇申請並檢視細節；3. 選擇「核准」或「拒絕」；4. 填寫審核備註（選填）；5. 系統更新申請狀態
後置條件	申請狀態更新為「已核准」或「已拒絕」

UC-03 登錄保養紀錄（管理員）

管理員選擇特定車輛並進入保養紀錄頁面，填寫保養日期、項目、費用與下次保養里程或日期，系統儲存紀錄，並於到期前在儀表板顯示提醒。

四、物件導向設計原則應用

本系統架構嚴格遵循以下 OOD 原則進行設計，以確保高內聚、低耦合，並具備未來擴充的彈性。

4.1 SOLID 原則

單一職責原則 (SRP)

Service 層依職責分離為三個獨立服務，每個類別只有單一修改理由：

- `BorrowingService`：只負責借車申請工作流程的業務邏輯。
- `VehicleService`：只負責車輛資產的 CRUD 與狀態管理。
- `MaintenanceService`：只負責保養紀錄的新增與到期提醒。

開閉原則 (OCP)

角色系統以 `Role` 介面為基礎，新增角色（如未來的「主管審核員」）只需實作 `Role` 介面並注入系統，不需修改既有程式碼。`BorrowingState` 的各狀態類別亦符合此原則：新增狀態只需實作介面，不需修改 `BorrowingRequest`。

依賴反轉原則 (DIP)

所有 Service 依賴抽象的 Repository 介面（`IVehicleRepository`、`IBorrowingRepository`），而非直接依賴具體的資料庫實作。如此一來，測試時可注入 `InMemoryVehicleRepo`，正式環境注入 `PostgresVehicleRepo`，Service 層完全不感知底層實作。

4.2 迪米特法則 (Law of Demeter)

借車流程狀態的轉換，透過 State Pattern 封裝於各個狀態類別內部。Service 層呼叫 `request.approve()` 即可，不需跨物件鏈式操作，有效降低元件間的知識耦合。

4.3 組合優於繼承

角色的權限設計採組合方式，`User` 物件持有 `Role` 介面的集合，`Role` 物件持有 `Permission` 介面的集合。未來若需要一個同時具備員工借車權限與部分審核權限的「主管」角色，只需組合對應的 `Permission`，不需建立複雜的繼承樹。

4.4 介面導向程式設計 (Program to an Interface)

系統核心邏輯中，所有跨邊界的依賴均以介面定義：

- `IVehicleRepository`：資料存取邊界
- `IBorrowingRepository`：借車資料存取邊界
- `BorrowingState`：流程狀態邊界
- `Role` / `Permission`：角色權限邊界

五、系統架構設計

5.1 分層架構

系統採五層式分層架構，各層職責明確且單向依賴：

層次	名稱	內容
第一層	Presentation Layer	REST API 端點、Vue 3 前端
第二層	Service Layer	BorrowingService、VehicleService、MaintenanceService
第三層	Domain Layer	BorrowingRequest (State Pattern)、Vehicle、User
第四層	Repository Interface	IVehicleRepository、IBorrowingRepository
第五層	Infrastructure Layer	PostgresVehicleRepo、PostgresBorrowingRepo

5.2 核心介面清單

IVehicleRepository

負責車輛資料的存取邊界，定義方法包含：依 ID 查詢單一車輛、依時段查詢可用車輛清單、儲存車輛資料、刪除車輛。

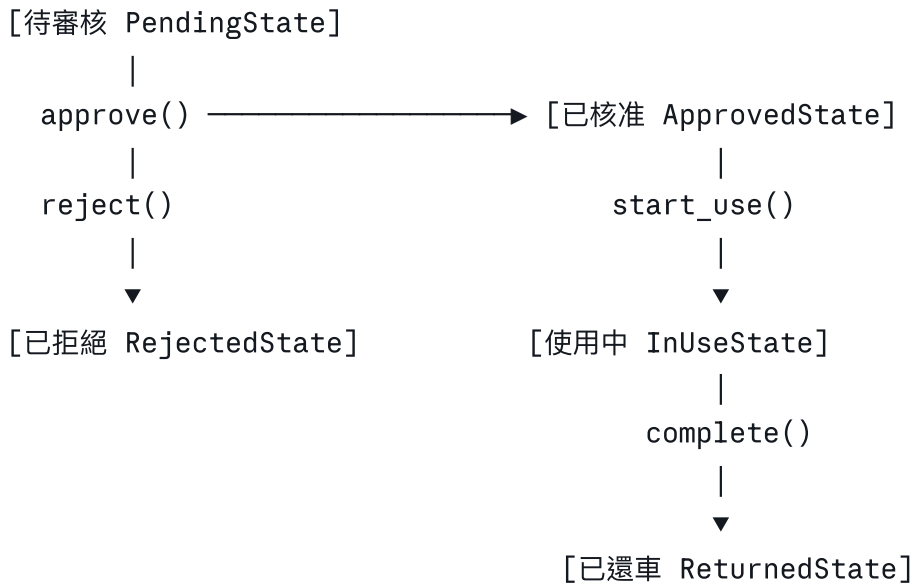
IBorrowingRepository

負責借車申請資料的存取邊界，定義方法包含：依 ID 查詢申請、查詢所有待審核申請、依使用者查詢申請歷程、儲存申請資料。

5.3 State Pattern：借車流程

借車申請的生命週期由 `BorrowingState` 介面的四個具體狀態類別管理，各狀態對相同操作的行為不同，非法的狀態轉換會拋出例外以防止資料一致性問題：

借車申請狀態流程：



5.4 角色與權限組合

角色與權限的對應關係以組合方式設計，避免繼承帶來的耦合問題：

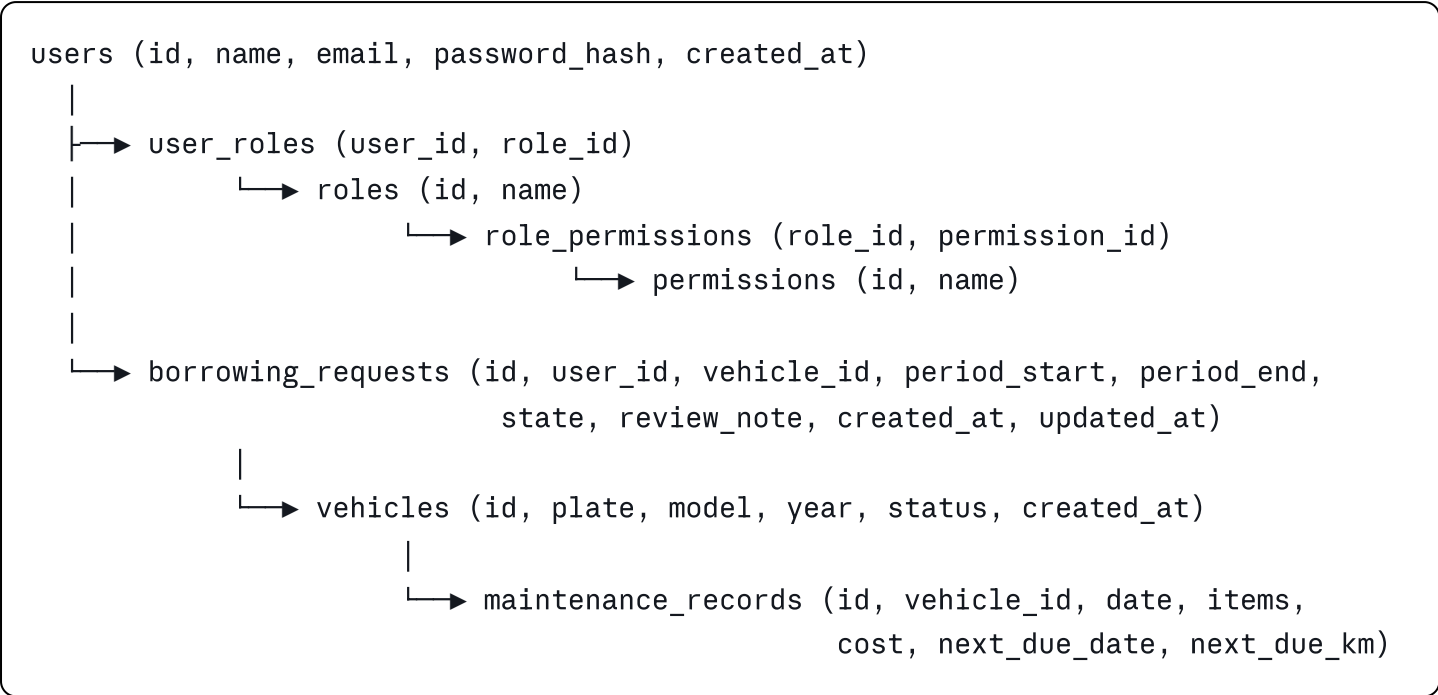
角色	持有權限
AdminRole	APPROVE_BORROWING、MANAGE_VEHICLE、SUBMIT_REQUEST
EmployeeRole	SUBMIT_REQUEST

`User` 持有一或多個 `Role`，透過 `can(permission_name)` 方法檢查操作是否被允許，Service 層不直接判斷角色類型，只詢問 `User` 是否具備特定 Permission。

5.5 Service 層職責邊界

`BorrowingService` 負責協調 `IBorrowingRepository` 與 `IVehicleRepository`，處理借車申請的提交、審核與完成流程。狀態轉換邏輯委託給 `BorrowingRequest` 物件（State Pattern），Service 不直接操作狀態字串，符合迪米特法則。

系統共設計九張資料表，核心關聯如下：



七、Repository 實作策略

依賴反轉原則使系統可在不同環境注入不同的 Repository 實作，而不需修改 Service 層：

- 正式環境注入 `PostgresVehicleRepo`，連接 PostgreSQL 資料庫。
- 測試環境注入 `InMemoryVehicleRepo`，以記憶體字典模擬資料庫，無需建立資料庫連線，測試執行速度可達毫秒級。

此策略使單元測試與外部基礎設施完全解耦，是 DIP 在實務上最直接的效益。

八、系統技術選型

層級	技術	選用理由
後端框架	FastAPI (Python)	原生支援 async、自動生成 OpenAPI 文件
資料庫	PostgreSQL	關聯完整性、ACID 保證
前端框架	Vue 3 (PWA)	漸進式 Web 應用，支援行動裝置
部署	VPS + Cloudflare DNS	穩定、低延遲

9.1 為何選擇 State Pattern 而非 Enum

借車流程的每個狀態不只是一個標籤，各狀態對同一操作的回應有本質差異。以「出車」操作為例，待審核狀態下應拋出例外，已核准狀態下才能合法轉換。State Pattern 將這些差異封裝於各狀態類別內，避免在 Service 層堆積大量條件判斷，使流程邏輯更易於維護與擴充。

9.2 為何 Role 以組合而非繼承實作

若以繼承方式設計角色階層，未來新增「只能審核但不能借車」的審計角色時，繼承鏈將無法乾淨地表達此語義，且可能引發多重繼承問題。組合方式允許任意 Permission 的組合，具備線性可擴充性，新增角色只需宣告其持有哪些 Permission 即可。

9.3 Repository 介面的測試價值

導入 `IVehicleRepository` 介面的直接效益是可測試性：單元測試中注入記憶體實作，毫秒內完成，且不依賴外部服務。這是 DIP 在實務上最具說服力的論據，也確保了未來更換資料庫（如從 PostgreSQL 改為其他方案）時，Service 層零修改。

十、專題分工

成員	負責範疇
王建葦	系統架構設計、OOD 原則應用、後端 API 實作
陳稚翔	前端 Vue 3 實作、資料庫設計、整合測試